Synchronization Markers for DNAFormer: Stage 4 Results

FYP, DNA data storage · 25 October 2026

Spacing synchronization markers through the strand raised fine-tuned DNAFormer's exact recovery at 8% deletion from **41.0%** to **52.4%**, against a control with identical payload and marker content.

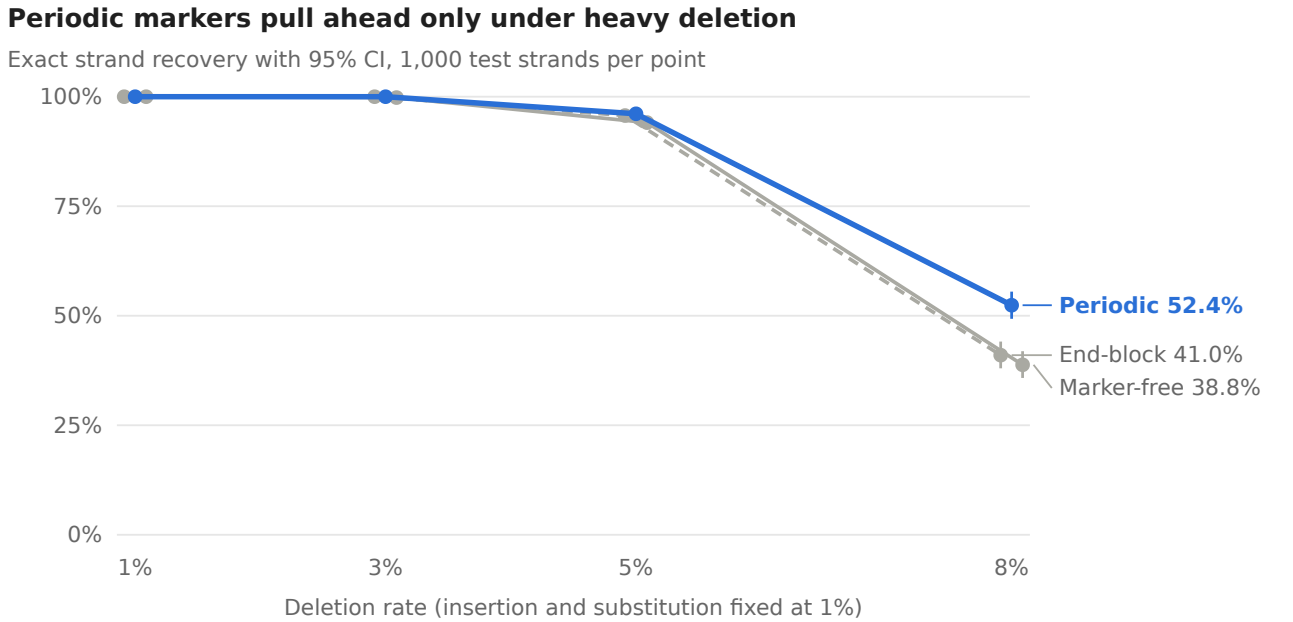
Question and setup

Do periodic markers (`ACGTCAGT` after every 24 data bases) help a fine-tuned DNAFormer recover 128-base strands when deletions are frequent?

Version	Layout of the 128 bases	Data bases
Marker-free	128 data	128
Periodic	4 × (24 data + 8 marker)	96
End-block (control)	96 data, then the 32 marker bases together	96

- **Data:** DNAFormer's own simulator; 10 noisy reads per strand; insertions and substitutions fixed at 1%; deletions at 1, 3, 5 and 8%. The versions share the same payloads and noise seeds.
- **Training:** each version fine-tuned from the same pretrained checkpoint with identical settings (10 epochs × 625 steps, best epoch by validation).
- **Scoring:** 1,000 unseen test strands per deletion level; markers removed, only data bases scored.

Results



Version	Data bases	1%	3%	5%	8%	Errors at 8% (NED)	Data recovered at 8%

							(bases per 1,000 strands)
Marker-free	128	100%	99.8%	94.1%	38.8%	0.020	49,664
Periodic markers	96	100%	100%	96.1%	**52.4%**	**0.015**	**50,304**
End-block control	96	100%	100%	95.7%	41.0%	0.024	39,360

Exact strand recovery of fine-tuned DNAFormer by deletion rate; 1,000 unseen test strands per level; markers removed before scoring.

Findings

1. **Marker spacing drives the gain.** Periodic and end-block carry the same 96 data and 32 marker bases; only placement differs. At 8% deletion periodic recovers 52.4% [49.3, 55.5] vs 41.0% [38.0, 44.1]: the intervals do not overlap.
2. **A shorter payload alone barely helps.** End-block vs marker-free at 8% is 41.0% vs 38.8% (overlapping intervals), and its edit distance is worse (0.024 vs 0.020).
3. **The effect appears only under heavy deletion.** All versions are near-perfect at 1–3%; at 5% they sit within 94–96% with overlapping intervals.
4. **Markers just pay for their 25% overhead.** At 8%, periodic recovers 50,304 data bases per 1,000 strands vs 49,664 marker-free (+1.3%).
5. **Fine-tuning itself helps every version.** At 8%, exact recovery rose from about 8% (pretrained) to 39–52%.

Is it a worthwhile finding?

Yes, for an FYP: it is a clean, controlled answer with an honest catch.

- **Why it counts:** the end-block control isolates synchronization from the shorter payload, so the claim is causal, not just "markers scored higher". It shows a classic coding idea (watermark-style markers, Davey and MacKay 2001) still helps a strong learned decoder.
- **The catch:** on stored data per strand the markers only break even (+1.3% at 8%), so they are not yet a practical win.
- **Limits:** simulated noise only; one marker sequence, one spacing (24) and one strand length; one training run per version; 10,000 training strands.
- **Before final claims:** Stage 5 adds paired tests on the same strands across versions and the final graphs. Worth testing next: other marker spacings or shorter markers, which could turn break-even into a net gain.

Fine-tuning settings

Identical for all three versions; only the dataset changed.

All three versions were fine-tuned from the same pretrained DNAFormer with DNAFormer's own optimizer and loss, seed 1, batch 16 and 10 epochs (2 with the transformer frozen, then 8 full).

Settings

Setting	Value	Why
Model	Pretrained DNAFormer (siamese), architecture unchanged, 103.4M parameters	Fine-tune, not redesign
Input / output	132 bases in, 128 out; 10 reads per strand (16 slots)	DNAFormer's original format
Random seed	**1**	Same as DNAFormer's code; makes runs repeatable
Optimizer	**Adam** (β1 0.9, β2 0.999, ε 1e-8, no weight decay)	Same as original DNAFormer
Loss	Cross-entropy + 0.25 × consistency	DNAFormer's own loss
Batch size	**16**	Fits a free T4 GPU (3.6 GB)
Epochs	**10** × 625 steps = 6,250 steps	Same fixed budget for every version
Phase A (epochs 1–2)	Transformer frozen, 2.6M parameters train; learning rate **1e-5** constant	Adapts the input side to deletion-heavy reads safely
Phase B (epochs 3–10)	Whole model trains; 100-step warm-up to **3e-6**, cosine decay to **1e-7**	Original 1e-5 scaled to batch 16 (vs 64)
Stability	Gradient clipping 1.0; mixed precision (fp16)	Avoids unstable jumps; faster on T4
Reads used	All 10 reads kept (relaxed filter)	Original filter drops most deletion-heavy reads

Data and selection

Item	Value
Training data	10,000 strands, 1/3/5/8% deletion mixed equally
Validation data	1,000 strands (mixed); picks the best epoch
Test data	1,000 strands per deletion level; used once, at the end
Best epoch chosen by	Lowest validation edit distance (data bases only)
Best epoch	Marker-free 9 · Periodic 10 · End-block 10
Hardware and time	Google Colab T4 GPU, about 35–40 minutes per version

Compared with original DNAFormer training

	Original DNAFormer	Our fine-tuning
Start	From scratch	Pretrained checkpoint
Data	1.5 million simulated strands per epoch	10,000 strands
Epochs / batch	100 / 64	10 / 16
Learning rate	1e-5 → 1e-7 (cosine)	1e-5 (frozen phase), then 3e-6 → 1e-7 (cosine)
Optimizer and loss	Adam, CE + 0.25 consistency	Same